Task 05 - Part 2 of 2: SQL Aggregations & GROUP BY

Course: Database Applications Development
Lesson: 05 - SQL Aggregations, Grouping, and Excel Export (in Part 2)

Instructions

Complete all exercises in this notebook. You will:

1. Write SQL queries using aggregate functions
2. Use GROUP BY to analyze categories
3. Export 4 query results to Excel files
4. Answer analysis questions

Resources:

- Lesson materials (dbApps05_AggregationsGrouping.md)
- Walkthrough notebook (dbApps05_Walkthrough.ipynb)
- SQL Reference Guide (updated with aggregations)

Submission:

1. Complete all TODO sections
2. Verify all cells run without errors
3. Check that Excel files were created (in Part 2)
4. Push to GitHub: `databaseApplications/dbApps05TasksPart2.ipynb`

Let's practice aggregations!

Setup

In [25]:
```
import pandas as pd
import sqlite3

# Connect to database
conn = sqlite3.connect('nba_5seasons.db')
print("✅ Connected to database")
```

```
✅ Connected to database
```

Part 3: HAVING Clause (4 queries)

Practice filtering groups with HAVING.

Remember:

- WHERE filters rows BEFORE grouping
- HAVING filters groups AFTER aggregating
- HAVING uses aggregate functions in the condition

Query 19: High-Scoring Teams

Task: Show teams that averaged over 110 points per game in 2021-22.

Hint: GROUP BY team, HAVING AVG(pts) > 110

In [6]:
```
# TODO: Write your query
query_19 = """
SELECT
    t.full_name AS team,
    ROUND(AVG(tgs.pts), 1) AS avg_points
FROM team_game_stats tgs
JOIN teams t ON tgs.team_id = t.team_id
WHERE tgs.season = '2021-22'
GROUP BY t.team_id, t.full_name
HAVING AVG(tgs.pts) > 110
ORDER BY avg_points DESC
"""

result_19 = pd.read_sql(query_19, conn)
display(result_19)
```

	team	avg_points
0	Minnesota Timberwolves	115.9
1	Memphis Grizzlies	115.6
2	Milwaukee Bucks	115.5
3	Charlotte Hornets	115.3
4	Phoenix Suns	114.8
5	Atlanta Hawks	113.9
6	Utah Jazz	113.6
7	San Antonio Spurs	113.2
8	Brooklyn Nets	112.9
9	Denver Nuggets	112.7
10	Los Angeles Lakers	112.1
11	Boston Celtics	111.8
12	Chicago Bulls	111.6
13	Indiana Pacers	111.5
14	Golden State Warriors	111.0
15	Sacramento Kings	110.3
16	Miami Heat	110.0

Query 20: Winning Teams

Task: Show teams that won at least 40 games in 2021-22.

Hint: Use SUM(CASE...) for wins, then HAVING count >= 40

In [9]:
```
# TODO: Write your query
query_20 = """
SELECT
    t.full_name AS team,
    SUM(CASE WHEN tgs.wl = 'W' THEN 1 ELSE 0 END) AS wins
FROM team_game_stats tgs
JOIN teams t ON tgs.team_id = t.team_id
WHERE tgs.season = '2021-22'
GROUP BY t.team_id, t.full_name
HAVING wins >= 40
ORDER BY wins DESC
"""

result_20 = pd.read_sql(query_20, conn)
display(result_20)
```

	team	wins
0	Phoenix Suns	64
1	Memphis Grizzlies	56
2	Golden State Warriors	53
3	Miami Heat	53
4	Dallas Mavericks	52
5	Boston Celtics	51
6	Milwaukee Bucks	51
7	Philadelphia 76ers	51
8	Utah Jazz	49
9	Denver Nuggets	48
10	Toronto Raptors	48
11	Chicago Bulls	46
12	Minnesota Timberwolves	46
13	Cleveland Cavaliers	44
14	Brooklyn Nets	44
15	Atlanta Hawks	43
16	Charlotte Hornets	43
17	Los Angeles Clippers	42

Query 21: Full Season Teams

Task: Show only teams that played at least 70 games in 2021-22.

Hint: GROUP BY team, HAVING COUNT(*) >= 70

In [12]:
```
# TODO: Write your query
query_21 = """
SELECT
    t.full_name AS team,
    COUNT(*) AS games_played
FROM team_game_stats tgs
JOIN teams t ON tgs.team_id = t.team_id
WHERE tgs.season = '2021-22'
GROUP BY t.team_id, t.full_name
HAVING games_played >= 70
ORDER BY games_played DESC
"""
```

```
result_21 = pd.read_sql(query_21, conn)
display(result_21)
```

	team	games_played
0	Atlanta Hawks	82
1	Boston Celtics	82
2	Cleveland Cavaliers	82
3	New Orleans Pelicans	82
4	Chicago Bulls	82
5	Dallas Mavericks	82
6	Denver Nuggets	82
7	Golden State Warriors	82
8	Houston Rockets	82
9	Los Angeles Clippers	82
10	Los Angeles Lakers	82
11	Miami Heat	82
12	Milwaukee Bucks	82
13	Minnesota Timberwolves	82
14	Brooklyn Nets	82
15	New York Knicks	82
16	Orlando Magic	82
17	Indiana Pacers	82
18	Philadelphia 76ers	82
19	Phoenix Suns	82
20	Portland Trail Blazers	82
21	Sacramento Kings	82
22	San Antonio Spurs	82
23	Oklahoma City Thunder	82
24	Toronto Raptors	82
25	Utah Jazz	82
26	Memphis Grizzlies	82
27	Washington Wizards	82
28	Detroit Pistons	82
29	Charlotte Hornets	82

Query 22: States with Multiple Teams

Task: Show states that have more than 2 NBA teams.

Hint: GROUP BY state, HAVING COUNT(*) > 2

In [15]:
```
# TODO: Write your query
query_22 = """
SELECT state, COUNT(*) AS teams_in_state
FROM teams
GROUP BY state
HAVING teams_in_state > 2
ORDER BY teams_in_state DESC, state
"""

result_22 = pd.read_sql(query_22, conn)
display(result_22)
```

	state	teams_in_state
0	California	4
1	Texas	3

Part 4: Export Queries to Excel (4 exports)

You will export 4 different query results to Excel files. These files will be used in the Excel task.

Export Guidelines:

- Use descriptive filenames
- Set index=False
- Use meaningful sheet names
- Verify files are created before submitting

Export 1: Team Performance Summary

Task: Export comprehensive team statistics for all teams in 2021-22.

Include: team name, city, games played, wins, losses, avg points, season high

Export to: `team_performance.xlsx`

In [29]:
```
# TODO: Write query for team performance
query_export_1 = """
SELECT
    t.full_name AS team,
    t.city,
```

```
    t.state,
    COUNT(tgs.game_id) AS games_played,
    ROUND(AVG(tgs.pts), 1) AS avg_points,
    MIN(tgs.pts) AS season_low,
    MAX(tgs.pts) AS season_high
FROM teams t
JOIN team_game_stats tgs ON t.team_id = tgs.team_id
WHERE tgs.season = '2021-22'
GROUP BY t.team_id, t.full_name, t.city, t.state
ORDER BY avg_points DESC
"""

# Execute and export
export_1 = pd.read_sql(query_export_1, conn)

# Display what we're exporting
print("Team Performance Data:")
display(export_1.head())

# Export to Excel
export_1.to_excel('team_performance.xlsx', index=False, sheet_name='Team Sta
print("✅ Exported to team_performance.xlsx")
```

```
Team Performance Data:
```

	team	city	state	games_played	avg_points	season_low	season_
0	Minnesota Timberwolves	Minnesota	Minnesota	82	115.9	84	
1	Memphis Grizzlies	Memphis	Tennessee	82	115.6	85	
2	Milwaukee Bucks	Milwaukee	Wisconsin	82	115.5	90	
3	Charlotte Hornets	Charlotte	North Carolina	82	115.3	86	
4	Phoenix Suns	Phoenix	Arizona	82	114.8	95	

```
✅ Exported to team_performance.xlsx
```

Export 2: Top Scorers

Task: Export the top 50 scoring players from 2021-22.

Include: player name, team name, games played, total points, points per game

Filter: Only players who played at least 20 games

Export to: `top_scorers.xlsx`

In [32]:
```
# TODO: Write query for top scorers
query_export_2 = """
```

```
SELECT
    p.full_name AS player_name,
    t.full_name AS team_name,
    ps.gp AS games_played,
    ROUND(ps.gp * ps.pts, 0) AS total_points,
    ps.pts AS points_per_game
FROM player_season_stats ps
JOIN players p ON ps.player_id = p.player_id
JOIN teams t   ON ps.team_id = t.team_id
WHERE ps.season = '2021-22'
  AND ps.gp >= 20
ORDER BY total_points DESC
LIMIT 50
"""

export_2 = pd.read_sql(query_export_2, conn)
print("Top Scorers Data:")
display(export_2.head(10))

export_2.to_excel('top_scorers.xlsx', index=False, sheet_name='Scoring Leade
print("✅ Exported to top_scorers.xlsx")
```

```
Top Scorers Data:
```

	player_name	team_name	games_played	total_points	points_per_game
0	Trae Young	Atlanta Hawks	76	163780.0	2155.0
1	DeMar DeRozan	Chicago Bulls	76	160968.0	2118.0
2	Jayson Tatum	Boston Celtics	76	155496.0	2046.0
3	Nikola Jokić	Denver Nuggets	74	148296.0	2004.0
4	Joel Embiid	Philadelphia 76ers	68	141372.0	2079.0
5	Karl-Anthony Towns	Minnesota Timberwolves	74	134532.0	1818.0
6	Giannis Antetokounmpo	Milwaukee Bucks	67	134134.0	2002.0
7	Miles Bridges	Charlotte Hornets	80	129040.0	1613.0
8	Devin Booker	Phoenix Suns	68	123896.0	1822.0
9	Luka Dončić	Dallas Mavericks	65	120055.0	1847.0

```
✅ Exported to top_scorers.xlsx
```

Export 3: Win-Loss Records

Task: Export win-loss records for all teams in 2021-22.

Include: team name, games played, wins, losses, win percentage (calculated)

Sort by: Most wins first

Export to: `win_loss_records.xlsx`

In [34]:
```
# TODO: Write query for win-loss records
query_export_3 = """
SELECT
    t.full_name AS team_name,
    COUNT(tgs.game_id) AS games_played,
    SUM(CASE WHEN tgs.wl = 'W' THEN 1 ELSE 0 END) AS wins,
    SUM(CASE WHEN tgs.wl = 'L' THEN 1 ELSE 0 END) AS losses
FROM team_game_stats tgs
JOIN teams t ON tgs.team_id = t.team_id
WHERE tgs.season = '2021-22'
GROUP BY t.team_id, t.full_name
ORDER BY wins DESC
"""

export_3 = pd.read_sql(query_export_3, conn)
export_3['win_pct'] = (export_3['wins'] / export_3['games_played'] * 100).rc

print("Win-Loss Records:")
display(export_3.head(10))

export_3.to_excel('win_loss_records.xlsx', index=False, sheet_name='Records'
print("✅ Exported to win_loss_records.xlsx")
```

```
Win-Loss Records:
```

	team_name	games_played	wins	losses	win_pct
0	Phoenix Suns	82	64	18	78.0
1	Memphis Grizzlies	82	56	26	68.3
2	Golden State Warriors	82	53	29	64.6
3	Miami Heat	82	53	29	64.6
4	Dallas Mavericks	82	52	30	63.4
5	Boston Celtics	82	51	31	62.2
6	Milwaukee Bucks	82	51	31	62.2
7	Philadelphia 76ers	82	51	31	62.2
8	Utah Jazz	82	49	33	59.8
9	Denver Nuggets	82	48	34	58.5

```
✅ Exported to win_loss_records.xlsx
```

Export 4: High-Scoring Games

Task: Export all games where a team scored 120+ points in 2021-22.

Include: team name, game date, opponent (from matchup), points scored, win/loss

Sort by: Points scored (highest first)

Export to: `high_scoring_games.xlsx`

In [37]:
```
# TODO: Write query for high-scoring games
query_export_4 = """
SELECT
    t.full_name AS team_name,
    tgs.game_date,
    tgs.matchup AS opponent,
    tgs.pts AS points_scored,
    tgs.wl AS win_loss
FROM team_game_stats tgs
JOIN teams t ON tgs.team_id = t.team_id
WHERE tgs.season = '2021-22'
  AND tgs.pts >= 120
ORDER BY tgs.pts DESC, tgs.game_date ASC
"""

export_4 = pd.read_sql(query_export_4, conn)
print("High-Scoring Games (120+ points):")
display(export_4.head(10))

export_4.to_excel('high_scoring_games.xlsx', index=False, sheet_name='High S
print("✅ Exported to high_scoring_games.xlsx")
```

```
High-Scoring Games (120+ points):
```

	team_name	game_date	opponent	points_scored	win_loss
0	Charlotte Hornets	2022-01-26	CHA @ IND	158	W
1	San Antonio Spurs	2022-02-25	SAS @ WAS	157	W
2	Washington Wizards	2022-02-25	WAS vs. SAS	153	L
3	Los Angeles Clippers	2022-04-01	LAC @ MIL	153	W
4	Memphis Grizzlies	2021-12-02	MEM vs. OKC	152	W
5	Brooklyn Nets	2022-03-15	BKN @ ORL	150	W
6	Minnesota Timberwolves	2022-03-14	MIN @ SAS	149	W
7	Houston Rockets	2021-11-27	HOU vs. CHA	146	W
8	Los Angeles Lakers	2022-04-10	LAL @ DEN	146	W
9	Boston Celtics	2021-12-04	BOS @ POR	145	W

```
✅ Exported to high_scoring_games.xlsx
```

Part 5: Analysis Questions

Answer these questions based on your queries. Write answers in the markdown cells below each question.

Question 1: Aggregate Functions

When would you use COUNT(*) vs COUNT(column)? Give a specific example from NBA data.

Your answer here:

Question 2: GROUP BY Purpose

Explain the difference between calculating average points WITHOUT GROUP BY vs WITH GROUP BY team_id.

Your answer here:

Question 3: WHERE vs HAVING

You want to find teams that averaged over 100 points, but only counting home games. Would you use WHERE, HAVING, or both? Explain why.

Your answer here:

Question 4: Real-World Application

Describe a business scenario (not sports) where you would use GROUP BY with aggregate functions.

Your answer here:

Question 5: Excel Integration

Why would you export SQL query results to Excel instead of keeping everything in the database?

Your answer here:

Verification Checklist

Before submitting, verify:

- [ ] All 22 queries completed and working
- [ ] All 4 Excel files created successfully
- [ ] All analysis questions answered
- [ ] All cells run without errors

- ☐ Meaningful column names used (AS aliases)
- ☐ Saved as `dbApps05TaskTrackA.ipynb`

Excel files that should exist:

1. team_performance.xlsx
2. top_scorers.xlsx
3. win_loss_records.xlsx
4. high_scoring_games.xlsx

Next: Open these files in the Excel task!

Clean Up

In [ ]:
```
# Close database connection
conn.close()
print("✅ Connection closed")
print("\nGreat work! Ready for the Excel task.")
```